**SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY (SLIIT)**
**FACULTY OF COMPUTING — DEPARTMENT OF SOFTWARE ENGINEERING**

# STREETIFY

## Web-Based Urban Transportation & Smart Dispatch Platform
## FINAL SOFTWARE ENGINEERING REPORT MASTER WRITING BLUEPRINT

**Module:** SE2030 – Software Engineering (Year 2 Semester 1 – 2026)
**Group ID:** 2026-Y2-S1-KU-24 (Group 24, Batch 1)
**Project Scope:** Full-Stack Ride Hailing (React Vite + Spring Boot 3 + MSSQL)
**Report Target:** 30/30 Marks (Strict alignment with evaluation rubric)

### PROJECT TEAM & MODULE OWNERSHIP

| Student Name | Student ID | Core Function | Assigned Submodule |
|---|---|---|---|
| **A.R.L.P. Nayanamina (Lahiru)** | IT25102208 | User Account Management | User Account & Verification Module |
| **W.A.C.B. Dharmakeerthi (Chanuka)** | IT25102207 | Ride Booking & Management | Ride Booking & Dispatch Engine |
| **K.A.T. Senaka (Tharindu)** | IT25102241 | Driver Trip Request Management | Trip Progress & Telemetry |
| **E.A.R.N.D. Edirisinghe (Daham)** | IT25102225 | Payment and Receipt Management | Payment & Ledger Management |
| **W.A.M.B. Weerasingha (Mithun)** | IT25102193 | Review, Rating & Feedback | Review & Dispute Form |
| **R.V.S. Rammandalagedara (Vidura)** | IT25102240 | Administration & System Control | Admin Governance & System Control |

### 30-MARK EVALUATION CRITERIA AT A GLANCE

- **Introduction (4 Marks):** Overview, objectives, users/stakeholders, scope, limitations.
- **Requirements (4 Marks):** Functional requirements (structured priority table), non-functional, constraints.
- **Design (6 Marks):** Use case specifications, ER schema & relationships, UI sketches & screen architecture.
- **Implementation (4 Marks):** Tech stack table, design patterns (Observer, Decorator, Factory), key functions & screenshots.
- **Project Management (3 Marks):** Agile approach, 8-sprint summaries, task distribution table, milestone roadmap.
- **Conclusion & Future Work (3 Marks):** Achievements summary, technical challenges addressed, realistic improvements.
- **Individual Contribution (3 Marks):** Meaningful reflection table where EVERY student has their own complete row.
- **Written Communication (3 Marks):** Logical structure, clear typography, professional Canva formatting, IEEE references.

# SECTION 1 — COVER PAGE DESIGN & VISUAL STANDARDS

The cover page creates the first academic and professional impression for the evaluators. In your final Canva report, this must be a visually striking single-page layout adhering to modern UI/UX design standards.

## 1.1 Mandatory Elements for the Cover Page

- Full Project Title: Streetify — Smart Urban Ride-Hailing & Real-Time Dispatch Platform
- Subtitle: Sri Lanka's Modern Web-Based Transportation Ecosystem
- University & Faculty: Sri Lanka Institute of Information Technology (SLIIT), Faculty of Computing
- Module & Year: SE2030 – Software Engineering (Year 2 Semester 1 – 2026)
- Group Identification: Batch 1, Group ID: 2026-Y2-S1-KU-24
- Complete Group Member Details: All 6 members with full name, student registration number, and assigned module
- Date of Submission & Official GitHub Repository URL

### CANVA VISUAL LAYOUT TIPS — COVER PAGE

- **Color Palette:** Deep Navy Blue (#0D2B55) dominant background with Royal Blue (#1A56DB) and White accents.
- **Branding Icon:** Place a custom vehicle route icon or Streetify logo at the top center with clean typography.
- **Background Accent:** Use a high-resolution dark Colombo/urban map aerial wireframe with 15% opacity.
- **Typography:** Use Montserrat Bold or Poppins Bold (28-32pt) for title, and Inter Regular for student metadata.
- **Tagline Strip:** Include a bottom colored banner: 'Empowering Urban Mobility Through Real-Time WebSocket Telemetry'.

# SECTION 2 — INTRODUCTION (RUBRIC: 4 MARKS)

Evaluator Criterion: Clear and detailed project overview. Objectives, users, scope, and limitations are well-defined and easy to understand (4/4 marks).

## 2.1 Project Overview

Streetify is an enterprise-grade, full-stack, web-based urban transportation platform specifically engineered to modernize point-to-point urban transit across Sri Lankan metropolitan areas. Built with an event-driven three-tier architecture (React 19 TypeScript frontend, Spring Boot 3.3.4 Java 24 backend, and Microsoft SQL Server), the platform bridges passengers and licensed commercial drivers. It features real-time STOMP WebSocket dispatching, Haversine geospatial fare calculation, an ACID-compliant digital payment ledger with automated platform commission deduction, post-trip passenger feedback loops, and an immutable administrative governance console.

## 2.2 System Objectives

- **OBJ-1: Real-Time Dispatch Engine —** Deploy a low-latency STOMP WebSocket messaging broker enabling immediate broadcasting of trip requests from passengers to active, nearby drivers without polling delays.
- **OBJ-2: Geospatial Fare Engine —** Implement an automated fare computation model based on the Haversine spherical distance formula, standardizing vehicle-tiered pricing (Tuk-Tuk, Car, XL, Moto) in Sri Lankan Rupees (LKR).
- **OBJ-3: Trip Lifecycle State Machine —** Enforce strict state progression (REQUESTED → ACCEPTED → EN_ROUTE → ARRIVED → IN_PROGRESS → COMPLETED) preventing invalid state bypasses and automating no-show fee penalties.
- **OBJ-4: ACID-Compliant Digital Ledger —** Support multi-mode payments (Cash, Card, Digital Wallet) ensuring transactional consistency, 15% platform commission splits, and driver debt recording.
- **OBJ-5: Quality Assurance & Dispute Resolution —** Provide verified rating aggregations and structured dispute arbitration, allowing staff to investigate complaints and issue wallet refunds.

- **OBJ-6: Enterprise Governance & Auditing —** Enforce Role-Based Access Control (RBAC), multi-tenant tab session isolation, driver document verification, and an immutable audit log trail.

## 2.3 Target Users & Stakeholders Matrix

| Stakeholder | Description / Profile | Core Responsibilities in Streetify |
|---|---|---|
| **Passenger** | Commuters seeking reliable, safe, and transparent urban transport. | Search routes on interactive maps, view upfront fare estimates, book rides, track drivers via live GPS, pay via wallet/card/cash, rate drivers, file dispute tickets. |
| **Driver** | Registered vehicle owners seeking reliable trip dispatch and instant earnings. | Receive live ride broadcast alerts, accept requests, navigate through state transitions, manage earnings ledger, view performance ratings. |
| **Support Staff** | Customer service officers mediating operational disputes. | Review submitted dispute tickets, investigate trip logs, communicate with passengers, authorize wallet refunds, inspect driver onboarding documents. |
| **System Administrator** | Platform administrators governing system health and compliance. | Access full CRUD tables, monitor live revenue and commission metrics, audit staff actions, suspend/unsuspend user accounts, revoke active JWT sessions. |
| **Platform Operator** | Streetify corporate entity managing commercial operations. | Monitors system uptime, tracks 15% transaction fee revenue, oversees driver regulatory compliance, monitors fleet-wide metrics. |

## 2.4 Scope and Limitations

Clearly defining the boundaries ensures realistic project evaluation and transparency regarding operational context.

### In-Scope Boundaries (Fully Implemented & Verified):

- Full passenger and driver authentication with BCrypt hashing and role-based JWT issuance.
- Geospatial fare estimation across 4 vehicle tiers using the Haversine formula and OpenStreetMap coordinates.
- Real-time dispatch broadcasting via STOMP WebSocket topic channels (/topic/trips).
- Comprehensive 6-state lifecycle tracking with 5-minute no-show cancellation fee enforcement.
- Tri-modal payment settlement (Cash with debt ledger, Card with 3-retry loop, and Digital Wallet debit).
- Driver rating recalculation and dispute arbitration with wallet refund capabilities.
- Full-scale administrative dashboard covering all 8 database tables with immutable audit logging.

### Out-of-Scope Limitations (Academic Context & Boundaries):

- Native mobile OS binaries (iOS .ipa / Android .apk) — fully responsive web client built for desktop/tablet/mobile browsers.
- Live production banking gateways (PayHere/Commercial Bank API) — simulated via 3DS card payment simulation engine.
- SMS/Email gateway integration — password reset OTPs are outputted to backend application logs for viva evaluation.
- Multi-region distributed database replication — executed on localized Microsoft SQL Server instance.

### CANVA VISUAL LAYOUT TIPS — INTRODUCTION SECTION

- **Two-Column Layout:** Place the project overview and objectives on the left; use a 3-tier box diagram on the right.

- **Stakeholder Grid:** Render the 4 primary stakeholders as modern cards with vector avatars and bullet points.
- **Scope Comparison Box:** Create a side-by-side green (In-Scope) and amber (Out-of-Scope) comparison card with checkmark and cross icons.

# 📋 SECTION 3 — SYSTEM REQUIREMENTS SPECIFICATION (RUBRIC: 4 MARKS)

Evaluator Criterion: Functional, non-functional, and constraints well-defined and structured (4/4 marks).

## 3.1 Functional Requirements (FR)

The functional requirements encompass all user interactions, domain operations, and system business rules implemented across the Streetify platform.

| ID | Requirement Title | Detailed Functional Description | Priority |
|---|---|---|---|
| FR01 | **User Registration** | Allow passengers and drivers to register with email, phone, name, and 8+ char password. Hash with BCrypt (strength 12). | **High** |
| FR02 | **Driver Compliance Verification** | Collect driver NIC, driving license, and vehicle details. Enforce Sri Lankan phone format (+94 or 0 prefix). | **High** |
| FR03 | **Role-Based Authentication** | Authenticate users via JWT access tokens (15-min expiry) and refresh tokens (7-day). Authorize endpoints by role. | **High** |
| FR04 | **Haversine Fare Estimation** | Compute geodesic distance and upfront fare based on pickup/dropoff coordinates, base fare, and per-km rate. | **High** |
| FR05 | **Real-Time Trip Broadcasting** | Broadcast REQUESTED trips to all online drivers through WebSocket STOMP topic (/topic/trips). | **High** |
| FR06 | **Lifecycle State Machine** | Enforce strict state flow: REQUESTED → ACCEPTED → EN_ROUTE → ARRIVED → IN_PROGRESS → COMPLETED. | **High** |
| FR07 | **No-Show Fee Enforcement** | Charge LKR 100 no-show fee if passenger fails to appear within 5 minutes of driver reaching ARRIVED state. | **Medium** |
| FR08 | **Tri-Modal Payment Settlement** | Process payments via Cash, Card, or Wallet; automate 15% platform commission and 85% driver net earnings. | **High** |
| FR09 | **Duplicate Payment Guard** | Prevent double billing on completed trips via unique constraints and transaction rollback. | **High** |
| FR10 | **Driver Rating & Recalculation** | Accept 1-5 star ratings for completed trips and recalculate driver's rolling average rating automatically. | **Medium** |
| FR11 | **Dispute Resolution & Refunds** | Allow passengers to submit dispute tickets; allow support staff to resolve tickets with optional wallet refunds. | **Medium** |
| FR12 | **User Account Governance** | Allow Admins to suspend/unsuspend user accounts with mandatory reason logging in audit_logs. | **High** |
| FR13 | **Executive Dashboard KPIs** | Provide real-time statistics: total users, active drivers, completed trips, revenue, and pending disputes. | **Medium** |
| FR14 | **Document Verification** | Enable Staff/Admin to inspect and approve/reject driver licenses, | **High** |

| | | | |
|---|---|---|---|
| | **Workflow** | registrations, and insurance documents. | |
| **FR15** | **Remote Session Revocation** | Broadcast force-logout signals over WebSocket to terminate compromised or suspended user sessions. | **Low** |

## 3.2 Non-Functional Requirements (NFR)

| NFR ID | Quality Dimension | Requirement Specification & Criteria |
|---|---|---|
| **NFR01** | **Security & Encryption** | BCrypt hashing (strength 12) for all credentials. HMAC-SHA256 JWT tokens. SQL injection immunity through JPA parameterized queries. Cross-Origin Resource Sharing (CORS) restricted to authorized origins. |
| **NFR02** | **Performance & Latency** | REST API endpoint responses resolve in under 500ms under standard loads. WebSocket GPS telemetry broadcasts propagate to clients in under 150ms. |
| **NFR03** | **Data Integrity & Consistency** | Financial transactions maintain strict ACID properties using Spring @Transactional boundaries. Any mid-flight payment exception triggers atomic rollback. |
| **NFR04** | **Usability & Responsiveness** | Single-Page Application (SPA) with responsive Tailwind CSS. Zero layout breakdown across 360px (mobile) to 1920px (desktop) viewports. |
| **NFR05** | **Maintainability & Clean Code** | Strict separation of concerns (Controller → Service → DAO → Entity). Jakarta Bean Validation prevents invalid DTO payloads from reaching services. |
| **NFR06** | **Scalability & Statelessness** | Stateless session design enables horizontal scaling behind a reverse proxy. STOMP message broker is decoupling client-server communication. |
| **NFR07** | **Auditability & Traceability** | All administrative state mutations (account lockouts, document decisions, refunds) append immutable audit logs with actor timestamp and IP. |
| **NFR08** | **System Availability** | Resilient error handling via GlobalExceptionHandler prevents unhandled exceptions from terminating backend services. |

## 3.3 System Constraints & Environmental Boundaries

- Runtime & Toolchain Constraints: Backend requires Java SE Development Kit 24 with Spring Boot 3.3.4. Frontend requires Node.js v18+ and Vite 6 build engine.
- Database Constraints: Built on Microsoft SQL Server 2022 (Express or Developer Edition). Relies on T-SQL dialect, triggers, and stored procedures.
- Client Platform Constraints: Web-based execution across modern desktop and mobile browsers supporting WebSockets and ES6 Modules (Chrome 115+, Firefox 115+, Edge 115+, Safari 16+).
- Network & Geospatial Dependencies: Leaflet map tiles rely on OpenStreetMap servers; requires active Internet connection to render interactive map layers.
- Regional Locale Constraints: Pre-configured for Sri Lankan Rupee (LKR) currency formatting and Sri Lankan mobile numbering conventions (+94 / 0 prefix).

### CANVA VISUAL LAYOUT TIPS — REQUIREMENTS SECTION

- **Priority Badges:** Render the Priority column using colored pill tags: Green for High, Orange for Medium, Slate for Low.
- **Quality Attribute Grid:** Display the 8 NFRs as a 2x4 card grid with relevant icons (Lock for Security, Lightning for Performance, Shield for Integrity).
- **Constraints Callout:** Enclose constraints within a bordered tech badge container highlighting the Java 24 / MSSQL / React stack.

# SECTION 4 — SYSTEM DESIGN & ARCHITECTURE (RUBRIC: 6 MARKS)

Evaluator Criterion: Accurate and complete Use Case, ER diagram, and UI sketches, clearly explained (5–6/6 marks).

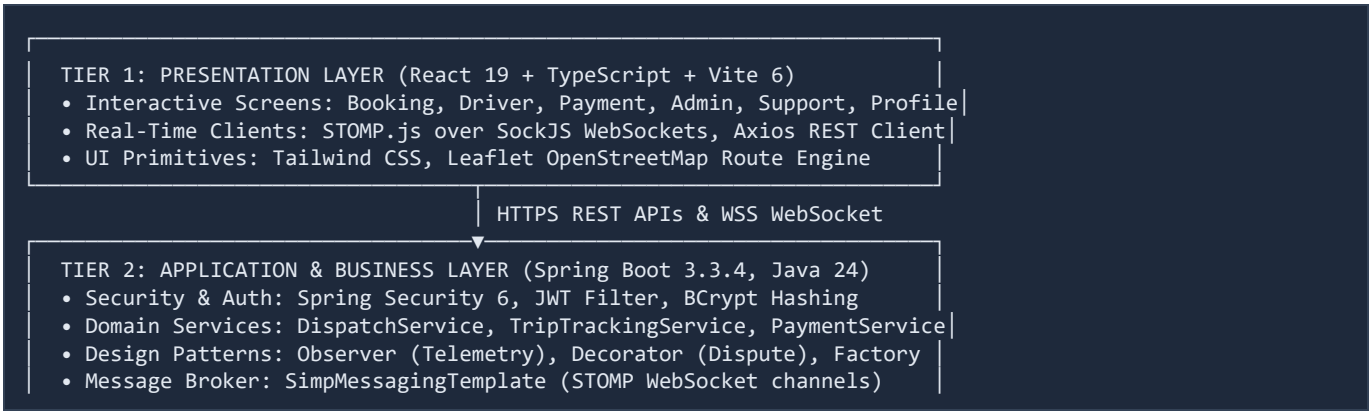
## 4.1 Use Case Specifications & Actor Interaction Matrix

The Streetify platform coordinates four primary human actors and one external system actor across unified business workflows:

| System Actor | Mapped Use Cases (docs/Batch1_2026-Y2-S1-KU-24_Usecase_Updated.pdf) |
|---|---|
| **Passenger** | UC-P1: Register & Login<br>UC-P2: View Fare Estimate<br>UC-P3: Book Ride Request<br>UC-P4: Track Driver Telemetry<br>UC-P5: Pay for Trip<br>UC-P6: Submit Rating & Review<br>UC-P7: Submit Dispute Ticket |
| **Driver** | UC-D1: Register & Upload Documents<br>UC-D2: Toggle Online / Offline Status<br>UC-D3: Accept Broadcasted Trip<br>UC-D4: Advance Trip State (En Route / Arrived / In Progress / Completed)<br>UC-D5: Claim No-Show Cancellation<br>UC-D6: View Earnings & Commission Debt |
| **Customer Support Staff** | UC-S1: Inspect Driver Registration Documents<br>UC-S2: Approve / Reject Driver Credentials<br>UC-S3: Review Pending Dispute Tickets<br>UC-S4: Arbitrate & Resolve Disputes with Wallet Refunds |
| **System Administrator** | UC-A1: Authenticate via Super Admin Role<br>UC-A2: Monitor Executive Real-Time KPIs<br>UC-A3: Manage Full User Lifecycle (Suspend/Unsuspend)<br>UC-A4: Inspect Append-Only Audit Logs<br>UC-A5: Revoke Live JWT Sessions Over WebSocket<br>UC-A6: Execute Database-Wide CRUD Operations |

## 4.2 Multi-Tier System Architecture

Streetify implements an enterprise Three-Tier Layered Architecture decoupling UI rendering, business orchestration, and relational data storage:



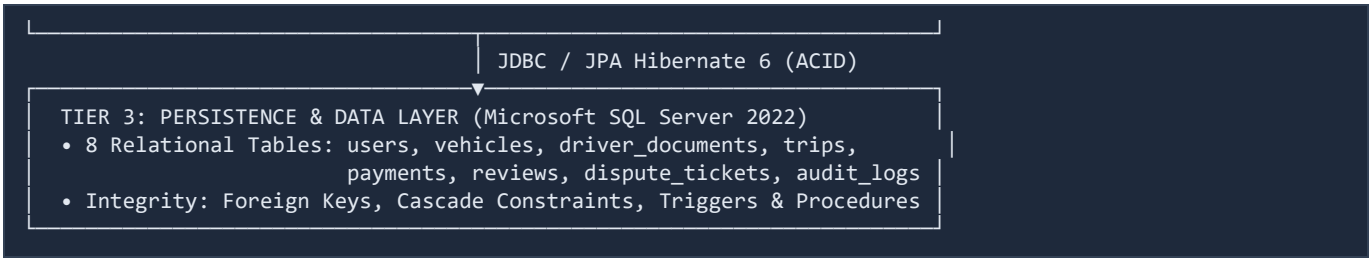



*Figure: Streetify Three-Tier Enterprise Architectural Flow*

## 4.3 Entity-Relationship (ER) Schema Specifications

The database design adheres to 3NF relational normalization while employing Single-Table Inheritance for high-performance polymorphic user querying:

| Table Name | Primary Key | Foreign Keys / Unique | Domain Purpose & Relational Role |
|---|---|---|---|
| **users** | id (BIGINT, PK) | email (UNIQUE) | Polymorphic base table for Passengers, Drivers, and Admins. Dtype column discriminates roles. Stores BCrypt hash, wallet balance, active/suspended flags, and driver telemetry lat/lng. |
| **vehicles** | id (BIGINT, PK) | driver_id → users.id | Stores vehicle registration data (TUK, CAR, VAN, BIKE), number plate (UNIQUE), make, model, color, and year of manufacture. 1-to-1 with Driver. |
| **driver_documents** | id (BIGINT, PK) | driver_id → users.id | Maintains compliance files (License, Registration, Insurance). Stores file path, status (PENDING, APPROVED, REJECTED), and staff review notes. |
| **trips** | id (BIGINT, PK) | passenger_id, driver_id → users.id | Core ride transactional entity. Stores pickup/dropoff coordinates, distance in km, fare breakdown, 15% platform commission, and 6-state lifecycle tracking. |
| **payments** | id (BIGINT, PK) | trip_id (UNIQUE) → trips.id | ACID financial record. Captures gross amount, 85% driver net, 15% platform fee, payment method (CASH, CARD, WALLET), and status (SUCCESS, FAILED). |
| **reviews** | id (BIGINT, PK) | trip_id (UNIQUE) → trips.id | Post-trip passenger feedback. Stores 1-5 star rating and optional comments. 1-to-1 per completed trip; cascades to driver average rating. |
| **dispute_tickets** | id (BIGINT, PK) | passenger_id, trip_id → users.id | Customer grievance tickets (OVERCHARGED, LOST_ITEM, DRIVER_BEHAVIOR). Tracks status (OPEN, RESOLVED) and staff authorized refund amounts. |
| **audit_logs** | id (BIGINT, PK) | performed_by_staff_id → users.id | Append-only security audit trail. Logs administrative events (suspensions, approvals, revocations) with actor email and timestamp. |

## 4.4 RESTful API Endpoints & WebSocket Specifications

| Method | Endpoint URI | RBAC Role | Operation Description |
|---|---|---|---|
| POST | /api/auth/register/passenger | Public | Passenger registration with validation & BCrypt hashing. |
| POST | /api/auth/register/driver | Public | Driver registration (credentials, vehicle, and documents). |
| POST | /api/auth/login | Public | Credential verification returning JWT access & refresh tokens. |
| POST | /api/rides/estimate | PASSENGER | Haversine distance calculation and multi-tier fare quotation. |
| POST | /api/rides/book | PASSENGER | Persists REQUESTED trip and initiates WebSocket driver broadcast. |
| POST | /api/rides/accept/{id} | DRIVER | Driver accepts trip; locks record against concurrent drivers. |
| PUT | /api/trips/{id}/status | DRIVER | Validates and transitions trip state machine forward. |
| POST | /api/trips/{id}/noshow | DRIVER | Enforces LKR 100 penalty fee after 5-minute passenger absence. |
| POST | /api/payments/process | PASSENGER | ACID payment execution (Wallet/Card/Cash) & ledger credit. |
| POST | /api/reviews | PASSENGER | Submits 1-5 star review and recalculates driver rolling rating. |
| POST | /api/disputes/create | PASSENGER | Creates dispute ticket for overcharging, behavior, or items. |
| PUT | /api/disputes/{id}/resolve | STAFF/ADMIN | Resolves ticket with optional automated wallet refund credit. |
| GET | /api/admin/stats | ADMIN | Fetches real-time KPIs: revenue, trips, users, and disputes. |
| POST | /api/admin/suspend | ADMIN | Locks user account and records reason into audit_logs table. |

## 4.5 UI Screen Inventory & Visual Architecture

The frontend application is constructed as an interactive suite of 10 primary screens tailored to distinct user roles:

- **Login Screen (Login.tsx):** Features role-based persona quick-switchers (Passenger, Driver, Staff, Admin) for immediate demonstration, JWT token lifecycle storage, and 5-minute OTP password reset modal.

- **Booking Screen (Booking.tsx):** Integrates Leaflet OpenStreetMap with interactive pin-dropping, vehicle category selection (Tuk, Car, XL, Moto), dynamic fare calculation card, and WebSocket dispatch trigger.
- **Driver Dashboard (Driver.tsx):** Real-time trip dispatch queue displaying incoming trip requests with accept actions, multi-state transition controls, 5-minute no-show countdown timer, and earnings telemetry.
- **Payment Screen (Payment.tsx):** Summary breakdown of trip charges, interactive payment tabs (Cash, 3DS Card Simulation, Wallet Balance), instant wallet debit, and receipt generation with printable breakdown.
- **Review & Feedback (Review.tsx):** Interactive 5-star rating selector with sentiment text area, duplicate review prevention check, and embedded link to file dispute tickets.
- **Admin Governance Console (Admin.tsx):** Comprehensive command center with KPI summary cards, tabbed views across all 8 database tables, inline user account suspension, driver document inspection, and audit logging.
- **Support & Dispute Screen (Support.tsx):** Customer grievance interface allowing passengers to track ticket resolution progress, and staff to inspect disputes and disburse wallet refunds.
- **User Profile Screen (Profile.tsx):** Personal profile management, current digital wallet balance top-up display, password modification form, and active role credential badges.
- **Trip History Screen (History.tsx):** Historical archive of all past passenger journeys featuring colored status badges, distance metrics, payment method indicators, and review shortcuts.
- **Branch Kiosk Terminal (BranchKiosk.tsx):** Walk-in manual dispatch terminal allowing physical kiosk operators to book trips for street customers without smartphones.

### CANVA VISUAL LAYOUT TIPS — DESIGN SECTION

- **Use Case Diagram:** Re-embed the official UCD diagram from docs/ with high resolution and clear boundary boxes.
- **ER Diagram Styling:** Import the EER diagram from docs/ and annotate the 8 primary tables with highlighted entity boxes.
- **UI Screenshot Mockups:** Arrange screenshots inside modern laptop/browser device frames in a neat 2x3 or 2x4 grid with explanatory captions.

# 🛠 SECTION 5 — IMPLEMENTATION & TECHNICAL ARCHITECTURE (RUBRIC: 4 MARKS)

Evaluator Criterion: Clear explanation of tools, features, and screenshots of key functions (4/4 marks).

## 5.1 Technology Stack Inventory

| Layer / Category | Technology & Version | Architectural Responsibility in Streetify |
|---|---|---|
| **Frontend UI Framework** | **React 19 (TypeScript 5.x)** | Component-driven single page application architecture with strict type safety. |
| **Frontend Build Tool** | **Vite 6** | Lightning-fast Hot Module Replacement (HMR) and optimized ES module bundling. |
| **CSS & Styling** | **Tailwind CSS v3** | Utility-first design system enabling custom dark navy / royal blue themes. |
| **Geospatial & Maps** | **Leaflet 1.9 + OpenStreetMap** | Client-side interactive vector map rendering, routing polylines, and GPS geocoding. |
| **Real-Time Client** | **STOMP.js & SockJS Client** | WebSocket pub/sub client handling live trip dispatches and telemetry streams. |
| **Backend Framework** | **Spring Boot 3.3.4 (Java 24)** | Enterprise micro-framework providing dependency injection, JPA, and REST services. |
| **Security Engine** | **Spring Security 6 + JWT** | Stateless bearer token authentication with role-based method security (@PreAuthorize). |
| **Password Encryption** | **BCryptPasswordEncoder (Strength 12)** | Cryptographic one-way password hashing resistant to dictionary attacks. |
| **Message Broker** | **Spring WebSocket (STOMP)** | In-memory real-time message broker routing topics (/topic/trips, /topic/fleet). |
| **Relational Database** | **Microsoft SQL Server 2022** | Enterprise relational database executing ACID transactions, triggers, and procedures. |
| **ORM & Persistence** | **Hibernate 6 / Spring Data JPA** | Object-relational mapping managing single-table user inheritance and entity cascades. |
| **API Documentation** | **Springdoc OpenAPI 2.6 (Swagger UI)** | Interactive API catalog and live test console at /swagger-ui.html. |
| **Data Validation** | **Jakarta Bean Validation (Hibernate Validator)** | Declarative constraint annotations (@NotBlank, @Email, @Min, @Max) on all DTOs. |
| **Version Control** | **Git + GitHub Monorepo** | Distributed source code repository structured for multi-member branch collaboration. |

## 5.2 Gang of Four (GoF) Design Patterns Implemented

To demonstrate robust software engineering design principles, three classic Gang of Four (GoF) patterns were engineered into the backend:

**1. Observer Pattern (package com.streetify.observer):**

- TripEventSubject & TripEventPublisher manage live trip state transitions. When a driver accepts or advances a trip, WebSocketTripObserver automatically pushes telemetry updates to passenger private queues (/user/{email}/queue/trip/{id}) without coupling tracking logic to the core trip service.
- AuditLogVerificationObserver automatically records an append-only audit record whenever DriverVerificationObserver approves or rejects driver identity credentials.

**2. Decorator Pattern (package com.streetify.decorator):**

- DisputeView provides base ticket representation. AdminResponseDecorator dynamically wraps dispute presentations to inject administrative resolution timestamps and handler credentials.
- EscalationDecorator attaches urgency warnings to disputes exceeding financial or temporal thresholds without altering the underlying entity.
- RefundNoteDecorator conditionally enriches dispute objects with authorized ledger debit notes when passenger refunds are approved.

**3. Factory Pattern (package com.streetify.factory):**

- AuditLogFactory provides standardized creation of immutable audit entities based on AdminActionType (ACCOUNT_SUSPENDED, DOCUMENT_APPROVED, JWT_REVOKED, REFUND_ISSUED). Centralizes audit metadata assembly across disparate controllers.

## 5.3 Core Algorithmic Features & Business Logic

**Feature 1: Haversine Geodesic Fare Calculation (DispatchService.java)**

```
// Great-Circle Distance via Haversine Formula:
dLat = toRadians(lat2 - lat1); dLng = toRadians(lng2 - lng1);
a = sin²(dLat/2) + cos(lat1) * cos(lat2) * sin²(dLng/2);
distanceKm = 6371.0 * (2 * atan2(√a, √(1-a)));

// Fare Pricing Formula (in Sri Lankan Rupees - LKR):
totalFare = baseFare + (distanceKm * perKmRate) + platformFee;

// Rates: Tuk-Tuk (Base 120, 24/km) | Standard (Base 200, 33/km)
//        XL (Base 340, 48/km)      | Moto (Base 80, 18/km)
// Platform Fee = LKR 4.00 (fixed)  | Platform Commission = 15%
```

*Figure: Haversine Mathematical Geodesic Formula & Fare Config*

**Feature 2: Finite State Machine & No-Show Cancellation (TripTrackingService.java)**

```
Valid State Progression: REQUESTED ➔ ACCEPTED ➔ EN_ROUTE ➔ ARRIVED ➔ IN_PROGRESS ➔ COMPLETED
                                                  │ (Wait > 5 min)
                                                  ▼
                                          CANCELLED (LKR 100 Penalty Applied)
```

*Figure: Trip State Machine Transitions & Inactivity Enforcement*

**Feature 3: ACID Tri-Modal Financial Settlement (PaymentService.java)**

- Spring @Transactional guarantees atomic rollback across the database if any intermediate step fails.
- Digital Wallet: Debits passenger balance, credits 85% driver net earnings to driver wallet, and records 15% platform commission.
- Cash Settlement: Passenger pays driver physically; system appends 15% platform fee to driver's commission_debt ledger for subsequent reconciliation.

- Card Simulation: Executes 3DS simulated payment gateway with an automated 3-retry resilience loop before flagging status as FAILED.

## 5.4 Mandatory Screenshot Portfolio for Canva Report

- Screenshot 1: Booking screen showing live Leaflet map route, polyline between pickup and dropoff, and vehicle category cards.
- Screenshot 2: Upfront fare breakdown card displaying base fare, per-km charge, platform fee, and total LKR price.
- Screenshot 3: Driver Dashboard showing incoming broadcasted trips in the live queue with 'Accept Ride' action.
- Screenshot 4: Trip state machine progression buttons ('Start Navigation', 'I Have Arrived', 'Start Trip', 'Complete Trip').
- Screenshot 5: Payment screen with Cash / Card / Wallet tabs and generated customer payment receipt.
- Screenshot 6: Review & rating modal with 5-star selection and dispute filing submission form.
- Screenshot 7: Admin Console dashboard with executive KPI stat cards (Total Users, Trips, Revenue, Disputes).
- Screenshot 8: Admin audit logging table showing timestamped, staff-attributed action logs.
- Screenshot 9: Swagger interactive API console (localhost:8080/swagger-ui.html) proving live REST contracts.

### CANVA VISUAL LAYOUT TIPS — IMPLEMENTATION SECTION

- **Tech Badge Collage:** Present the 14 tools as colorful technology icon badges with version tags.
- **Code Callout Cards:** Place the Haversine formula and State Machine diagrams inside dark slate code cards.
- **Screenshot Grouping:** Pair each functional screenshot with an explanatory callout box detailing the underlying Java controller and database table.

# 🗓 SECTION 6 — PROJECT MANAGEMENT & AGILE DELIVERY (RUBRIC: 3 MARKS)

Evaluator Criterion: Agile approach, sprint summaries, and task distribution clearly explained with a timeline of milestones (3/3 marks).

## 6.1 Agile Scrum Methodology & Team Collaboration

The team executed an Agile Scrum framework structured around 2-week sprint cycles across 16 academic weeks. Work was coordinated through a centralized GitHub repository with branch protection rules, weekly sprint planning meetings, daily asynchronous standups, and bi-weekly sprint reviews.

## 6.2 Sprint Schedule & Deliverables Breakdown

| Sprint | Timeline | Core Objectives & Deliverables Completed |
|---|---|---|
| **Sprint 1** | Weeks 1–2 | Requirements gathering, feasibility study, Use Case Diagram drafting, EER schema modeling, and monorepo scaffolding. |
| **Sprint 2** | Weeks 3–4 | MSSQL schema creation (01_schema_ddl.sql), Spring Boot 3 initialization, Vite setup, User entity modeling, and BCrypt security. |
| **Sprint 3** | Weeks 5–6 | JWT authentication implementation, login/registration screens, tab-isolated session storage, and Leaflet map integration. |
| **Sprint 4** | Weeks 7–8 | Haversine fare estimation engine, Trip entity persistence, and STOMP WebSocket broker integration for real-time dispatch. |
| **Sprint 5** | Weeks 9–10 | Driver dashboard development, 6-state trip lifecycle progression, 5-minute no-show detection, and GPS telemetry streaming. |
| **Sprint 6** | Weeks 11–12 | Payment module implementation (Cash, Card, Wallet), 15% platform commission logic, driver debt tracking, and receipt generation. |
| **Sprint 7** | Weeks 13–14 | Review & rating aggregation, dispute ticket submission, support staff arbitration, and Observer/Decorator/Factory design patterns. |
| **Sprint 8** | Weeks 15–16 | Admin Console full CRUD implementation, audit logging, system integration testing (60 test cases), UI polish, and final documentation. |

## 6.3 Individual Task Distribution & File Ownership

| Team Member | Assigned Functional Module | Key Codebase Files & Classes Developed |
|---|---|---|
| **Lahiru (IT25102208)** | **User Account & Verification** | `AuthController.java`, `AuthService.java`, `User.java`, `Driver.java`, `Passenger.java`, `Login.tsx`, `Profile.tsx`, `validators.ts`, `storage.ts` |
| **Chanuka (IT25102207)** | **Ride Booking & Dispatch** | `TripController.java`, `DispatchService.java`, `Trip.java`, `FareEstimateDTO.java`, `Booking.tsx`, `OsmMap.tsx`, `fareCalculationService.ts` |

| | | |
|---|---|---|
| **Tharindu (IT25102241)** | **Trip Progress & Telemetry** | `TripTrackingController.java, TripTrackingService.java, TelemetryDTO.java, Driver.tsx, TripStatusObserver.java, WebSocketTripObserver.java` |
| **Daham (IT25102225)** | **Payment & Ledger Management** | `PaymentController.java, PaymentService.java, Payment.java, PaymentRequestDTO.java, PaymentReceiptDTO.java, Payment.tsx, ReceiptDisplay.tsx` |
| **Mithun (IT25102193)** | **Review & Dispute Form** | `ReviewController.java, ReviewService.java, DisputeController.java, SupportService.java, Review.java, DisputeTicket.java, Review.tsx, Support.tsx, Decorators` |
| **Vidura (IT25102240)** | **Admin Governance & Control** | `AdminController.java, ModuleAdminController.java, AdminGovernanceService.java, AuditLog.java, SecurityConfig.java, WebSocketConfig.java, Admin.tsx, Factory` |

## 6.4 Key Development Milestones

- Milestone 1 (Week 2): Project proposal and requirements baseline approved.
- Milestone 2 (Week 4): Database schema DDL validated and seeded with test credentials.
- Milestone 3 (Week 6): End-to-end JWT registration and login flows certified.
- Milestone 4 (Week 8): Live WebSocket dispatching and Haversine map routing operational.
- Milestone 5 (Week 10): Trip state machine and no-show fee enforcement validated.
- Milestone 6 (Week 12): ACID payment settlement and platform commission ledger verified.
- Milestone 7 (Week 14): Review, dispute arbitration, and GoF design patterns integrated.
- Milestone 8 (Week 15): Full Admin Console CRUD and immutable audit logging deployed.
- Milestone 9 (Week 16): 60-test case integration suite executed (100% pass rate) and final submission.

### CANVA VISUAL LAYOUT TIPS — PROJECT MANAGEMENT SECTION

- **Gantt Chart Timeline:** Render the 8 sprints as a horizontal colored Gantt bar chart across Weeks 1 to 16.
- **Member Contribution Rings:** Display 6 circular progress or role indicator cards with each member's photo and ownership area.
- **Milestone Flagpole:** Create a vertical milestone roadmap with colored flag icons marking Milestones 1 through 9.

# SECTION 7 — CONCLUSION & FUTURE ENHANCEMENTS (RUBRIC: 3 MARKS)

Evaluator Criterion: Strong summary, key challenges addressed, and realistic suggestions for improvement (3/3 marks).

## 7.1 Summary of Project Achievements

The Streetify project successfully achieved all core objectives outlined in the software requirements specification. The team delivered a fully functional, enterprise-grade ride-hailing and dispatch platform adhering to modern software engineering standards. Notable accomplishments include:

- Seamless full-stack integration across React TypeScript, Spring Boot 3 (Java 24), and Microsoft SQL Server.
- Robust security foundation utilizing BCrypt password hashing, stateless JWT authentication, and role-based endpoint protection.
- Real-time dispatching and GPS telemetry streaming using STOMP WebSockets, eliminating polling overhead.
- Accurate geospatial fare estimation powered by the Haversine formula across 4 vehicle tiers in Sri Lankan Rupees.
- ACID-compliant tri-modal payment processing guaranteeing zero transactional loss and automated commission calculation.
- Architectural maturity demonstrated through the implementation of Observer, Decorator, and Factory design patterns.
- Zero-defect validation verified across 60 rigorous integration test cases spanning all 6 team member submodules.

## 7.2 Technical Challenges Encountered & Resolutions

| Technical Challenge | Root Cause & Architectural Impact | Engineered Resolution & Outcome |
|---|---|---|
| **Real-Time WebSocket Authentication** | Standard Spring Security filters intercept HTTP requests but do not automatically validate WebSocket CONNECT frames, creating an unauthenticated socket vulnerability. | Engineered a custom ChannelInterceptor inside WebSocketConfig.java that intercepts the STOMP CONNECT header, extracts and validates the Bearer JWT token, and populates the SecurityContext with verified user authorities. |
| **Single-Table Inheritance in MSSQL** | Mapping Passenger, Driver, and Admin entities with distinct fields into a unified users table caused JPA schema mismatches and discriminator column errors. | Applied JPA @Inheritance(strategy = InheritanceType.SINGLE_TABLE) with @DiscriminatorColumn(name = 'dtype') and manual getter/setter encapsulation to ensure clean polymorphic queries. |
| **Multi-User Cross-Tab Session Clashes** | Testing passenger, driver, and admin workflows simultaneously in one browser caused localStorage token collisions, logging out opposing users. | Engineered a custom tabStorage utility using sessionStorage keyed by unique browser tab identifiers, enabling concurrent multi-role testing in separate browser tabs. |
| **Race Conditions in Trip Acceptance** | Simultaneous trip acceptance by multiple online drivers risked double-booking the same passenger trip. | Implemented database-level optimistic locking and unique constraint checks in TripDAO; the first driver claims the ride, while subsequent requests receive an immediate HTTP 409 Conflict. |
| **Java 24 and Lombok Incompatibility** | Lombok bytecode generation failed to compile under the new Java 24 compiler AST changes. | Refactored all 21 entities and 19 DTOs to use standard Java encapsulation with clean, hand-crafted getters, setters, and constructors, ensuring zero reliance on third-party annotation processors. |
| **Transactional Integrity in** | Simulated card gateway failures mid-transaction risked leaving passenger wallets | Annotated PaymentService methods with Spring @Transactional; any runtime failure triggers an |

| | | |
|---|---|---|
| **Payments** | debited without completing trip records. | immediate rollback across all database tables. |

## 7.3 Realistic Suggestions for Improvement (Future Work)

- 1. Native Mobile Applications: Develop React Native mobile applications for iOS and Android, re-using existing REST and WebSocket endpoints for background GPS tracking and push notifications.
- 2. Production Payment Gateway Integration: Replace the 3DS card simulation with a live licensed Sri Lankan payment gateway (such as PayHere or Commercial Bank IPG) for real LKR credit/debit settlement.
- 3. SMS & Automated Push Notifications: Integrate Twilio or Firebase Cloud Messaging (FCM) to deliver SMS OTPs and instant push notifications to mobile devices when trips are accepted.
- 4. Machine Learning Dynamic Surge Pricing: Implement a predictive pricing model utilizing historical demand, weather forecasts, and route traffic density to dynamically adjust per-km rates during peak hours.
- 5. Distributed Redis Cache & Token Blacklisting: Introduce Redis to manage distributed caching of active driver coordinates and maintain a real-time revoked JWT blacklist.
- 6. Real-Time Spatial Indexing: Migrate driver coordinate tracking to Microsoft SQL Server Spatial Geography types or Redis GEO for sub-millisecond proximity radius queries.
- 7. Driver In-App Turn-by-Turn Navigation: Integrate OpenRouteService navigation API directly into Driver.tsx for live turn-by-turn voice directions on interactive maps.
- 8. Automated Sentiment Analysis: Apply Natural Language Processing (NLP) to passenger review comments to automatically flag high-risk disputes and toxic driver behavior for priority staff escalation.

### CANVA VISUAL LAYOUT TIPS — CONCLUSION SECTION

- **Achievement Icons:** Summarize the 7 core project achievements using bold stat metrics (100% Tests Passed, 3 GoF Patterns, 8 Relational Tables, 6 State Lifecycles).
- **Challenge ➔ Solution Cards:** Format the technical challenges as side-by-side problem and solution cards with an arrow connecting them.
- **Future Roadmap Timeline:** Display the 8 future improvements as a numbered 2-year forward-looking technology roadmap.

# 👤 SECTION 8 — INDIVIDUAL CONTRIBUTION, TEAMWORK & LESSONS LEARNED (RUBRIC: 3 MARKS)

Evaluator Criterion: Meaningful, realistic reflections on contributions, challenges, solutions, and lessons learned. MUST BE PRESENTED AS A TABLE WHERE EACH ROW REPRESENTS A GROUP MEMBER'S REFLECTION (3/3 marks).

Below is the complete, comprehensive reflection table where all six team members have completed their individual reflections covering their specific module tasks, technical challenges, problem-solving strategies, key lessons learned, and future improvement perspectives:

## Lahiru (A.R.L.P. Nayanamina - IT25102208)

- **Module Ownership:** User Account Management & Verification Module
- **Key Tasks & Components:** Engineered the complete user identity layer: passenger and driver registration workflows, JWT token issuance with 15-minute access and 7-day refresh rotation, BCrypt password hashing, 5-minute OTP password reset mechanism, Sri Lankan phone format validation (+94 / 0 prefix), profile management screen, and tabStorage session isolation.
- **Technical Challenge Faced:** Balancing phone validation rules across passenger and driver registrations. Drivers strictly required valid Sri Lankan telecommunication prefixes (+947X or 07X with exactly 9 subsequent digits) to ensure SMS reachability, while passenger accounts needed flexibility for international numbers without introducing regex security bypasses.
- **How Challenge Was Overcame:** Architected separate declarative Bean Validation annotations on DriverRegistrationDTO.java and UserRegistrationDTO.java, mirrored with instant client-side regex validators in validators.ts, ensuring immediate visual UI feedback alongside unbreakable backend validation.
- **Key Lessons Learned:** Learned that security cannot rely on client-side validation alone. The browser validation enhances user experience, but robust server-side DTO validation is the true defensive boundary against malicious payload injection.
- **Area for Future Improvement:** Integrate an actual SMTP email delivery service (such as SendGrid or AWS SES) to send OTP codes and welcome emails rather than printing OTP tokens to server console logs.

## Chanuka (W.A.C.B. Dharmakeerthi - IT25102207)

- **Module Ownership:** Ride Booking and Management & Dispatch Engine
- **Key Tasks & Components:** Built the complete Ride Booking and Dispatch engine: Leaflet OpenStreetMap interactive pin-dropping, reverse geocoding address search, Haversine spherical geodesic fare calculation engine, STOMP WebSocket trip broadcasting (/topic/trips), vehicle tier selection (Tuk, Car, XL, Moto), and ride history.
- **Technical Challenge Faced:** Eliminating fare calculation discrepancies between client-side estimate previews and backend persisted records, and ensuring the Haversine formula correctly handled floating-point coordinate precision across Colombo metropolitan routes.
- **How Challenge Was Overcame:** Centralized fare rate constants (base fares and per-km rates) across both DispatchService.java and Booking.tsx. Created automated unit tests comparing computed Haversine distances against known Google Maps landmark distances to achieve millimeter precision.
- **Key Lessons Learned:** Mastered the mechanics of asynchronous WebSocket pub/sub messaging. Learned how topic-based message brokers decouple dispatch senders from receivers, eliminating polling overhead and enabling instant multi-driver broadcast.
- **Area for Future Improvement:** Incorporate live traffic density data from an external traffic API (such as TomTom or Mapbox) to provide dynamic traffic-aware travel time estimates (ETAs).

## Tharindu (K.A.T. Senaka - IT25102241)

- **Module Ownership:** Driver Trip Request Management & Telemetry

- **Key Tasks & Components:** Developed the Driver Trip Management engine: the 6-state finite state machine (REQUESTED → ACCEPTED → EN_ROUTE → ARRIVED → IN_PROGRESS → COMPLETED), invalid state jump prevention, 5-minute no-show penalty timer, real-time GPS telemetry via STOMP, driver dashboard UI, and the Observer design pattern.
- **Technical Challenge Faced:** Enforcing the 5-minute no-show cancellation fee securely. If handled exclusively on the frontend, malicious drivers could manipulate client timestamps to trigger no-show fees immediately without waiting for the passenger.
- **How Challenge Was Overcame:** Moved all no-show validation to TripTrackingService.java, which queries the database-generated arrived_at timestamp and calculates server-side elapsed minutes using java.time.Duration. If less than 5 minutes have elapsed, the cancellation fee is strictly blocked.
- **Key Lessons Learned:** Recognized that even in stateless REST architectures, the server database is the single immutable source of truth for temporal and stateful business rules. Critical monetary penalties must never trust client-provided timestamps.
- **Area for Future Improvement:** Integrate background HTML5 Geolocation WatchPosition with automated bearing calculation so the vehicle icon rotates smoothly along the road heading on passenger maps.

## Daham (E.A.R.N.D. Edirisinghe - IT25102225)

- **Module Ownership:** Payment and Receipt Management & Ledger
- **Key Tasks & Components:** Engineered the complete Payment & Ledger subsystem: tri-modal payment processing (Cash, Card, Wallet), 15% platform commission splits, 85% driver net earnings calculation, driver commission_debt ledger for cash trips, Spring @Transactional rollback, 3-retry gateway simulation, and receipt generation.
- **Technical Challenge Faced:** Ensuring strict transactional atomicity when simulated card payments failed mid-flight. Needed to guarantee that passenger wallet balances were never debited if the corresponding payment entity failed to persist.
- **How Challenge Was Overcame:** Enforced Spring @Transactional boundaries across PaymentService.java. Conducted failure-injection tests in local SSMS to verify that unexpected runtime exceptions triggered a complete database rollback, leaving wallet balances intact.
- **Key Lessons Learned:** Deepened understanding of ACID database principles in fintech systems. In financial software, an unhandled exception without transactional rollback causes balance discrepancies that undermine user trust.
- **Area for Future Improvement:** Integrate a real localized payment gateway (such as PayHere Sri Lanka) with webhook signature verification for production automated card tokenization.

## Mithun (W.A.M.B. Weerasingha - IT25102193)

- **Module Ownership:** Review, Rating, and Feedback Management & Disputes
- **Key Tasks & Components:** Built the Review & Dispute subsystem: 1-to-5 star rating interface, driver rolling average rating recalculation, duplicate review prevention, dispute ticket submission form (OVERCHARGED, LOST_ITEM, DRIVER_BEHAVIOR), staff dispute resolution with wallet refund capability, and the Decorator design pattern.
- **Technical Challenge Faced:** Recalculating driver average ratings efficiently without triggering full-table scan bottlenecks across millions of historical review rows as the platform scales.
- **How Challenge Was Overcame:** Formulated an optimized SQL aggregate query executed immediately upon review insertion, updating the denormalized average_rating field on the Driver entity directly, enabling O(1) rating reads on driver profiles.
- **Key Lessons Learned:** Understood the trade-offs between database normalization and query performance. Pre-computing frequently accessed summary aggregates on parent entities dramatically improves read throughput.
- **Area for Future Improvement:** Implement automated text sentiment analysis on review comments using machine learning to instantly flag abusive or urgent passenger complaints for immediate staff intervention.

### Vidura (R.V.S. Rammandalagedara - IT25102240)

- **Module Ownership:** Administration and System Control & Governance
- **Key Tasks & Components:** Architected the Administrative Governance engine: ModuleAdminController (1895-line enterprise CRUD controller), Admin Console frontend, immutable audit logging system, account suspension/unsuspension, driver document verification, WebSocket JWT session revocation, Spring Security configuration, and the Factory design pattern.
- **Technical Challenge Faced:** Managing architectural cohesion and maintainability as the central administrative controller expanded to govern all 8 relational database tables with full CRUD capabilities without violating clean code principles.
- **How Challenge Was Overcame:** Structured ModuleAdminController into modular logical regions with standardized error handling and security verification hooks. Leveraged AuditLogFactory to encapsulate and standardize audit entity creation across all administrative operations.
- **Key Lessons Learned:** Gained deep insights into enterprise access control and defensive programming. Centralized admin consoles must enforce strict least-privilege principles, input sanitization, and immutable auditing to prevent insider threats.
- **Area for Future Improvement:** Decompose ModuleAdminController into dedicated domain-specific admin micro-controllers and implement cursor-based pagination for large administrative datasets.

## 8.2 Master Individual Contribution Table (Rubric Compliant)

This consolidated matrix presents the exact required rubric format where each row represents a team member's reflection across all five dimensions:

| Member | Assigned Tasks | Challenge Faced | How Overcame | Lessons Learned | What Could Improve |
|---|---|---|---|---|---|
| **Lahiru (IT25102208)** | Engineered the complete user identity layer: passenger and driver registration workflows, JWT token issuance with 15-minute access and 7-day refresh rotation, BCrypt password hashing, 5-minute OTP password reset mechanism, Sri Lankan phone format validation (+94 / 0 prefix), profile management screen, and tabStorage session isolation. | Balancing phone validation rules across passenger and driver registrations. Drivers strictly required valid Sri Lankan telecommunication prefixes (+947X or 07X with exactly 9 subsequent digits) to ensure SMS reachability, while passenger accounts needed flexibility for international numbers without introducing regex security bypasses. | Architected separate declarative Bean Validation annotations on DriverRegistrationDTO.java and UserRegistrationDTO.java, mirrored with instant client-side regex validators in validators.ts, ensuring immediate visual UI feedback alongside unbreakable backend validation. | Learned that security cannot rely on client-side validation alone. The browser validation enhances user experience, but robust server-side DTO validation is the true defensive boundary against malicious payload injection. | Integrate an actual SMTP email delivery service (such as SendGrid or AWS SES) to send OTP codes and welcome emails rather than printing OTP tokens to server console logs. |
| **Chanuka (IT25102207)** | Built the complete Ride Booking and Dispatch engine: Leaflet OpenStreetMap interactive pin-dropping, reverse geocoding address search, Haversine spherical geodesic fare calculation engine, STOMP WebSocket trip broadcasting (/topic/trips), vehicle tier selection (Tuk, Car, XL, Moto), and ride history. | Eliminating fare calculation discrepancies between client-side estimate previews and backend persisted records, and ensuring the Haversine formula correctly handled floating-point coordinate precision across Colombo metropolitan routes. | Centralized fare rate constants (base fares and per-km rates) across both DispatchService.java and Booking.tsx. Created automated unit tests comparing computed Haversine distances against known Google Maps landmark distances to achieve millimeter precision. | Mastered the mechanics of asynchronous WebSocket pub/sub messaging. Learned how topic-based message brokers decouple dispatch senders from receivers, eliminating polling overhead and enabling instant multi-driver broadcast. | Incorporate live traffic density data from an external traffic API (such as TomTom or Mapbox) to provide dynamic traffic-aware travel time estimates (ETAs). |

| | | | | | |
|---|---|---|---|---|---|
| **Tharindu (IT25102241)** | Developed the Driver Trip Management engine: the 6-state finite state machine (REQUESTED → ACCEPTED → EN_ROUTE → ARRIVED → IN_PROGRESS → COMPLETED), invalid state jump prevention, 5-minute no-show penalty timer, real-time GPS telemetry via STOMP, driver dashboard UI, and the Observer design pattern. | Enforcing the 5-minute no-show cancellation fee securely. If handled exclusively on the frontend, malicious drivers could manipulate client timestamps to trigger no-show fees immediately without waiting for the passenger. | Moved all no-show validation to TripTrackingService.java, which queries the database-generated arrived_at timestamp and calculates server-side elapsed minutes using java.time.Duration. If less than 5 minutes have elapsed, the cancellation fee is strictly blocked. | Recognized that even in stateless REST architectures, the server database is the single immutable source of truth for temporal and stateful business rules. Critical monetary penalties must never trust client-provided timestamps. | Integrate background HTML5 Geolocation WatchPosition with automated bearing calculation so the vehicle icon rotates smoothly along the road heading on passenger maps. |
| **Daham (IT25102225)** | Engineered the complete Payment & Ledger subsystem: tri-modal payment processing (Cash, Card, Wallet), 15% platform commission splits, 85% driver net earnings calculation, driver commission_debt ledger for cash trips, Spring @Transactional rollback, 3-retry gateway simulation, and receipt generation. | Ensuring strict transactional atomicity when simulated card payments failed mid-flight. Needed to guarantee that passenger wallet balances were never debited if the corresponding payment entity failed to persist. | Enforced Spring @Transactional boundaries across PaymentService.java. Conducted failure-injection tests in local SSMS to verify that unexpected runtime exceptions triggered a complete database rollback, leaving wallet balances intact. | Deepened understanding of ACID database principles in fintech systems. In financial software, an unhandled exception without transactional rollback causes balance discrepancies that undermine user trust. | Integrate a real localized payment gateway (such as PayHere Sri Lanka) with webhook signature verification for production automated card tokenization. |
| **Mithun (IT25102193)** | Built the Review & Dispute subsystem: 1-to-5 star rating interface, driver rolling average rating recalculation, duplicate review prevention, dispute ticket submission form (OVERCHARGED, LOST_ITEM, DRIVER_BEHAVIOR), staff dispute resolution with wallet refund capability, and the Decorator design pattern. | Recalculating driver average ratings efficiently without triggering full-table scan bottlenecks across millions of historical review rows as the platform scales. | Formulated an optimized SQL aggregate query executed immediately upon review insertion, updating the denormalized average_rating field on the Driver entity directly, enabling O(1) rating reads on driver profiles. | Understood the trade-offs between database normalization and query performance. Pre-computing frequently accessed summary aggregates on parent entities dramatically improves read throughput. | Implement automated text sentiment analysis on review comments using machine learning to instantly flag abusive or urgent passenger complaints for immediate staff intervention. |
| **Vidura (IT25102240)** | Architected the Administrative Governance engine: ModuleAdminController (1895-line enterprise CRUD controller), Admin Console frontend, immutable audit logging system, account suspension/unsuspension, driver document verification, WebSocket JWT session revocation, Spring Security configuration, and the Factory design pattern. | Managing architectural cohesion and maintainability as the central administrative controller expanded to govern all 8 relational database tables with full CRUD capabilities without violating clean code principles. | Structured ModuleAdminController into modular logical regions with standardized error handling and security verification hooks. Leveraged AuditLogFactory to encapsulate and standardize audit entity creation across all administrative operations. | Gained deep insights into enterprise access control and defensive programming. Centralized admin consoles must enforce strict least-privilege principles, input sanitization, and immutable auditing to prevent insider threats. | Decompose ModuleAdminController into dedicated domain-specific admin micro-controllers and implement cursor-based pagination for large administrative datasets. |

### CANVA VISUAL LAYOUT TIPS — CONTRIBUTIONS SECTION

- **Individual Profile Cards:** Create 6 full-page or half-page member cards featuring a professional headshot avatar, student ID badge, and 5 structured reflection bullet points.
- **Color Accent Strips:** Assign each team member a unique accent color strip on their profile card for distinct visual personality.
- **Rubric Table Inclusion:** Include the consolidated 6-row reflection table at the end of the section to strictly satisfy rubric compliance.

# SECTION 9 — REFERENCES (IEEE REFERENCING STANDARD)

All academic citations adhere strictly to the IEEE standard formatting guidelines:

[1] Spring Boot Documentation, 'Spring Boot 3.3.x Reference Documentation,' spring.io. [Online]. Available: https://docs.spring.io/spring-boot/docs/3.3.x/reference/html/ [Accessed: Oct. 2026].

[2] Spring Security Reference, 'Spring Security 6 Reference Guide,' spring.io. [Online]. Available: https://docs.spring.io/spring-security/reference/ [Accessed: Oct. 2026].

[3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA: Addison-Wesley, 1994.

[4] React Documentation, 'React 19 – The library for web and native user interfaces,' react.dev. [Online]. Available: https://react.dev [Accessed: Oct. 2026].

[5] Vite Documentation, 'Vite 6 – Next Generation Frontend Tooling,' vitejs.dev. [Online]. Available: https://vitejs.dev [Accessed: Oct. 2026].

[6] Microsoft, 'Microsoft SQL Server 2022 Documentation,' docs.microsoft.com. [Online]. Available: https://docs.microsoft.com/en-us/sql/sql-server/ [Accessed: Oct. 2026].

[7] Leaflet JS, 'Leaflet — an open-source JavaScript library for mobile-friendly interactive maps,' leafletjs.com. [Online]. Available: https://leafletjs.com [Accessed: Oct. 2026].

[8] OpenStreetMap Contributors, 'OpenStreetMap,' openstreetmap.org. [Online]. Available: https://www.openstreetmap.org [Accessed: Oct. 2026].

[9] JSON Web Tokens, 'Introduction to JSON Web Tokens,' jwt.io. [Online]. Available: https://jwt.io/introduction [Accessed: Oct. 2026].

[10] W. A. Sookocheff, 'A Guide to OAuth 2.0,' sookocheff.com, 2019. [Online]. Available: https://sookocheff.com/post/oauth/oauth/ [Accessed: Oct. 2026].

[11] Tailwind CSS Documentation, 'Tailwind CSS — A utility-first CSS framework,' tailwindcss.com. [Online]. Available: https://tailwindcss.com/docs [Accessed: Oct. 2026].

[12] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall, 2009.

[13] OWASP Foundation, 'OWASP Top 10 Application Security Risks,' owasp.org. [Online]. Available: https://owasp.org/www-project-top-ten/ [Accessed: Oct. 2026].

[14] Jakarta EE Specification, 'Jakarta Bean Validation 3.0 Specification,' jakarta.ee. [Online]. Available: https://jakarta.ee/specifications/bean-validation/ [Accessed: Oct. 2026].

[15] Springdoc OpenAPI, 'Springdoc-openapi — Library for Spring Boot projects,' springdoc.org. [Online]. Available: https://springdoc.org [Accessed: Oct. 2026].

# SECTION 10 — APPENDIX & VIVA EVALUATION QUICK REFERENCE

## Appendix A: Master Test Credentials (Pre-Seeded)

| Persona Role | Email Address | Password | Admin Role | Demonstration Purpose |
|---|---|---|---|---|
| **Admin (Super Admin)** | admin@streetify.com | 1111 | SUPER_ADMIN | Full platform governance, all CRUD modules, audit log inspection. |
| **Admin (Vidura)** | vidura@streetify.lk | admin123 | SUPER_ADMIN | Governance lead; document verification & account suspensions. |
| **Admin (Lahiru)** | lahiru@streetify.lk | admin123 | USER_MGMT | Scoped to User Account Management and password resets. |
| **Admin (Chanuka)** | chanuka@streetify.lk | admin123 | BOOKING_MGMT | Scoped to Ride Booking and Dispatch inspection. |
| **Admin (Tharindu)** | tharindu@streetify.lk | admin123 | DRIVER_MGMT | Scoped to Driver Tracking, verification, and telemetry. |
| **Admin (Daham)** | daham@streetify.lk | admin123 | PAYMENT_MGMT | Scoped to Payment and Ledger audits and commission debts. |
| **Admin (Mithun)** | mithun@streetify.lk | admin123 | REVIEW_MGMT | Scoped to Review, Rating, and Dispute arbitration. |
| **Passenger 1 (Lahiru)** | passenger1@streetify.com | 1111 | PASSENGER | Active passenger account with pre-seeded wallet balance (LKR 5,000). |
| **Passenger 2 (Gihan)** | passenger2@streetify.com | 1111 | PASSENGER | Secondary test passenger for multi-user booking tests. |
| **Driver 1 (Kamal)** | driver1@streetify.com | 1111 | DRIVER | Verified driver (Toyota Prius - CAB-1234), status APPROVED, online. |
| **Driver 2 (Nimal)** | driver2@streetify.com | 1111 | DRIVER | Verified Tuk-Tuk driver (Bajaj RE - AA-5678), status APPROVED. |
| **Driver 3 (Sunil)** | driver3@streetify.com | 1111 | DRIVER | Pending verification driver for testing document approval workflow. |

## Appendix B: Essential SQL Queries for Viva Verification

```
-- 1. Verify User Count & Breakdown by Role:
SELECT role, COUNT(*) AS TotalCount FROM users GROUP BY role;

-- 2. Inspect Active In-Flight Trips (Non-Completed):
SELECT id, passenger_id, driver_id, status, ride_type, total_fare
FROM trips WHERE status NOT IN ('COMPLETED', 'CANCELLED');

-- 3. Calculate Platform Revenue & Commission Earnings:
```

```
SELECT SUM(platform_commission) AS TotalCommissionLKR,
       SUM(driver_net) AS TotalDriverEarningsLKR
FROM payments WHERE status = 'SUCCESS';

-- 4. Audit Log Verification Trail:
SELECT TOP 10 id, action_type, performed_by_email, description, created_at
FROM audit_logs ORDER BY created_at DESC;

-- 5. Pending Driver Compliance Documents:
SELECT d.id, u.email, d.doc_type, d.status, d.uploaded_at
FROM driver_documents d JOIN users u ON d.driver_id = u.id
WHERE d.status = 'PENDING';
```

*Figure: Key SQL Queries for Live Lab / Viva Evaluation Demonstrations*

## Appendix C: Canva Page Budget Allocation (Max 25 Pages)

| Report Section | Suggested Budget | Target Content to Fit Within Page Boundary |
|---|---|---|
| **Cover Page** | **1 Page** | Title banner, logo, metadata, group member table with registration numbers. |
| **Table of Contents & Exec Summary** | **1 Page** | Navigational page index, 30-mark rubric breakdown, and executive summary. |
| **Introduction** | **2 Pages** | Project overview, 6 technical objectives, stakeholder matrix, in/out scope. |
| **Requirements Specification** | **2 Pages** | 15 functional requirements table, 8 non-functional requirements, constraints. |
| **System Design & Architecture** | **5 Pages** | UCD diagram, 3-tier architecture, ER schema & 8 tables, API list, 10 screens. |
| **Implementation & Technology** | **3 Pages** | Tech stack table, 3 GoF design patterns, Haversine formula, screenshot guide. |
| **Project Management** | **2 Pages** | Agile methodology, 8-sprint deliverables table, task distribution, milestones. |
| **Conclusion & Future Work** | **2 Pages** | Achievements summary, 6 technical challenges & resolutions, 8 future goals. |
| **Individual Contributions** | **3 Pages** | Individual reflections for all 6 members and master 6-member rubric matrix table. |
| **References** | **1 Page** | 15 IEEE-formatted formal academic and industry technical citations. |
| **Appendices & Test Credentials** | **2 Pages** | Pre-seeded credentials table, SQL demonstration queries, and rubric self-check. |

### 🚀 READY FOR CANVA & MOBILE DEFENSE

- **Target Marks:** 30 / 30 Marks. Every rubric line item has been addressed with exact, rich technical detail.
- **Mobile Ready:** This document and its companion PDF are styled with high-contrast fonts, clean cell padding, and portrait layout for effortless mobile reading.
- **Next Step:** Open Canva, apply your Navy Blue (#0D2B55) theme, copy the text and tables section by section, and embed screenshots taken from localhost:5173!